

isis Técnico

Velasquez
Morales
/ Barrios

codigoo (Java)

```
---< com.mycompany:Analysis_tecnico_Java_Grafos.exe >-----
Analysis_tecnico_Java_Grafos.exe 1.0-SNAPSHOT
m.xml
-----[ jar ]-----
rces:3.3.1:resources (default-resources) @ Analysis_tecnico_Java_Grafos.exe ---
existing resourceDirectory C:\Users\Usuario\Documents\NetBeansProjects\Analysis_tecnico_Java_Grafos.exe
ler:3.13.0:compile (default-compile) @ Analysis_tecnico_Java_Grafos.exe ---
ng the module because of changed source code.
1 source file with javac [debug release 25] to target\classes

3.5.1:exec (default-cli) @ Analysis_tecnico_Java_Grafos.exe ---
3 5 4 6 7
5 6 4 3 7
CESS
e: 1.460 s
at: 2026-05-25T18:21:25-06:00
```

código (C++)

Analysis_tecnico_C++_Grafos.cpp

:

\\Users\\Usuario\\Desktop\\tarea-hash-csv\\Analysis_tecnico_C++_Grafos.ex

S: 1 3 2 4 5 6 7

S: 1 3 4 6 7 5 2

process finished with exit code 0

Estructuras se implementaron en C++?

pero las estructuras de nodos enlazados
adyacencia (NodoAdyacente y Vertice) y la
manejo de la Cola manual (NodoCola),
conexiones enteramente mediante

```
// 1. Estructura manual para enlazados
struct NodoAdyacente {
    int destino;
    NodoAdyacente* siguiente;
};

// 2. Estructura manual para representacion de grafos
struct Vertice {
    int id;
    NodoAdyacente* lista;
};

// 3. Estructura manual para enlazados de cola
struct NodoCola {
    int dato;
    NodoCola* siguiente;
};

// 4. Implementacion manual de la Cola
class Cola {
private:
    NodoCola* frente;
    NodoCola* final;
public:
    Cola() {
        frente = nullptr;
        final = nullptr;
    }
};
```

Java?

ovechamos todo lo que nos
ns Framework de Java, que ya
ras hechas. Para armar la
afo usamos un HashMap, y
o a los vecinos en listas tipo
bién nos sirvieron para anotar
s visitando. Por último, para el
lugar de programar la cola a
ros, simplemente usamos la
bajando en conjunto con una
ya trae de fábrica.

```
// 1. Importacion de las estructuras
import java.util.HashMap;
import java.util.ArrayList;
import java.util.Queue;
import java.util.LinkedList;

public class EstructurasNativasJava {

    // 2. Uso de HashMap y ArrayList
    private HashMap<Integer, ArrayList<Integer>>
        grafico = new HashMap<>();

    public EstructurasNativasJava() {
        grafico = new HashMap<>(); // Inicialización
    }

    public void recorridos() {
        // 3. Uso de Queue implementado con LinkedList
        Queue<Integer> cola = new LinkedList<>();

        // 4. Uso de ArrayList para la lista de vecinos
        ArrayList<Integer> visitados = new ArrayList<>();

        // Ejemplo de uso de sus métodos
        cola.add(1);
        visitados.add(1);
    }
}
```

largo?

C++ es largo. Esto se exige escribir la lógica de los nodos, la de las listas de gestión de los e y final de la e liberación de tructor.



4. ¿Cuál código es más fácil de entender?



Definitivamente el de Java que manejar punteros usar métodos que ya como `.add()`, `.putIfAbsent()` código queda mucho más fácil entender qué exactamente el BFS y el

control del programador?

vamente en C++. Como en esta práctica no usamos que ya te dan todo hecho, toca controlar cada pedacito. Primero, hay que tener muchísimo cuidado al trabajar con punteros, porque si pierdes un enlace, se te rompe la estructura del grafo. Y segundo, está el tema del manejo de memoria que destruyes tú mismo al terminar. En Java que limpia por ti, en C++ si se te olvida la memoria que usaste, se quedan esos datos consumidos por la computadora, lo que genera las famosas fugas de memoria.

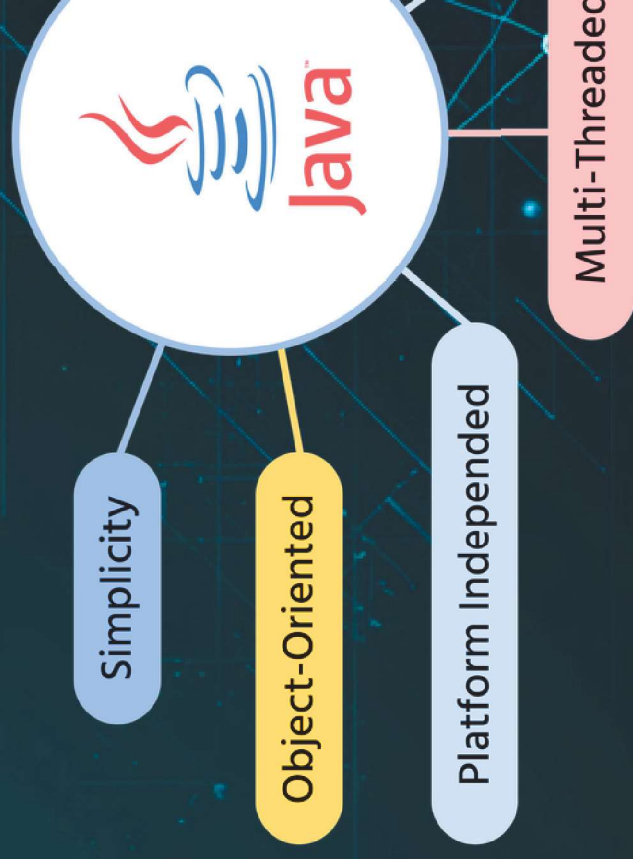
¿Ventajas tiene en este caso?

C++ en este caso es su eficiencia y el ahorro de recursos. Como no tiene un overhead en segundo plano ni objetos tan pesados, nos ahorramos memoria adicional (el famoso overhead). Esto hace que el programa consuma estrictamente la memoria que se le pide y se ejecute de una manera mucho más eficiente sobre la máquina.



en este caso?

Java de Java es la agilidad
y la seguridad que
estamos utilizando
datos que ya vienen
preparadas y optimizadas por
Java, es mucho menos
posibilidad de errores de lógica.
Debido a su recolector de
basura, nos quitamos de
preocupación de colapsar el sistema
por la memoria."

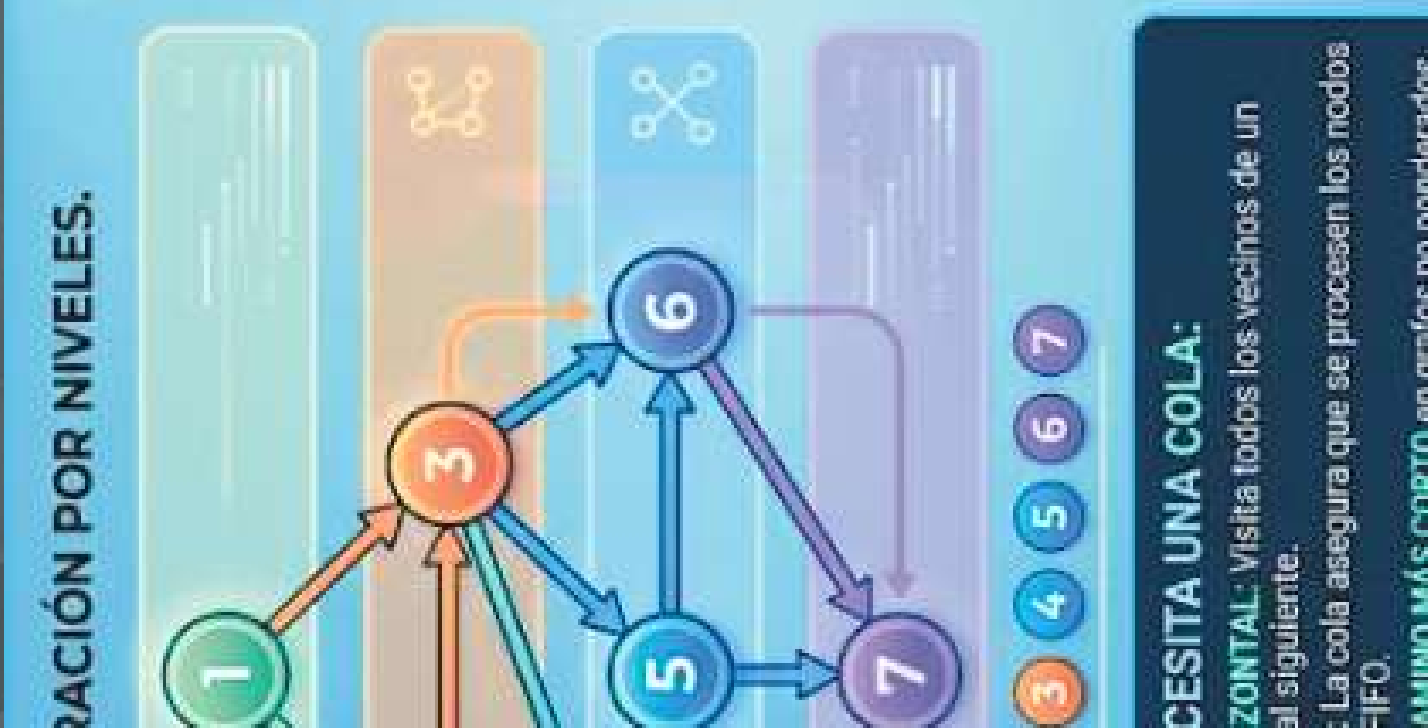


era memoria correctamente

El principal problema es que se generan fugas de memoria (memory leaks). Al crear los vértices y nodos, utilizamos la función new para reservar espacio en el sistema operativo, pero nunca nos liberar ese espacio al terminar, el sistema lo mantiene reservado creyendo que aún lo necesitamos. Aunque el sistema sea pequeño parezca inofensivo, en aplicaciones grandes consume la memoria RAM progresivamente, lo que termina provocando el rendimiento o provocando que el sistema colapse por completo.

9. ¿Por qué BFS necesita una cola?

El recorrido BFS requiere está diseñado para explorar niveles. Al utilizar una estructura FIFO (First In, First Out o Primero en Salir), la cola garantiza el orden correcto de la búsqueda: primero se procesen y muestren los vecinos inmediatos de un nodo antes de avanzar a explorar los nodos de mayor profundidad.



¿Cómo resolverse con recursión?

La ventaja de Java es la agilidad para desarrollar y la seguridad que estamos utilizando para manejar los datos que ya vienen optimizados y optimizadas por el lenguaje, es mucho menos propenso a cometer errores de lógica. Gracias a su recolector de basura automático, nos quitamos el peso de colapsar el sistema por la memoria."

